ESTRUCTURAS DE DATOS

ARREGLOS, VECTOR, COLA , PILA ,SET, MAP

Soluciones anterior contest

H Lexicographical_challenge

H_Lexicographical_Challenge.cpp > solve()

```cpp
#include <bits/stdc++.h>    #include errors detected

using namespace std ;
const int tam = 6 * 1e5;
int arr[tam];
void solve()
{
   string s ;
    int k ;
    cin >> s>> k ;
    int n = s.size();

    for(int i = 0 ; i < k ; i++)
    {
        vector<char> v;
        for(int j = i; j < n ; j+= k)
        {
            v.push_back(s[j]);
        }

        sort(v.begin(), v.end());

        for(int j = i , indice = 0 ; j < n ; j+= k ,
        {
            s[j] = v[indice];
        }
    }
    cout << s << '\n';
}

int main()
{
    ios::sync_with_stdio(0); cin.tie(0);
        solve();

}
```

J Escalator

J_Escalator.cpp > ...

```cpp
#include <bits/stdc++.h>    #include errors detected
using namespace std;

int dir[120000];

void solve()
{
    int n, time, d, now, last, wait;
    cin >> n;
    for (int i = 1; i ≤ 100200; i++)
    {
        dir[i] = -1;
    }
    for (int i = 1; i ≤ n; i++)
    {
        cin >> time >> d;
        dir[time] = d;
    }
    now = -1;
    last = 0;
    wait = -1;
    for (int i = 1; i ≤ 100020; i++)
    {
        if (dir[i] ≠ -1)
        {
            if (now == dir[i])
                last = i;
            else
            {
                wait = dir[i];
            }
        }
```

```cpp
        if (last + 10 == i)
        {
            now = -1;
        }
        if (now == -1 && wait ≠ -1)
        {
            now = wait;
            last = i;
            wait = -1;
        }
    }
    last += 10;
    cout << last;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    solve();
}
```

I Eliminatin ballons

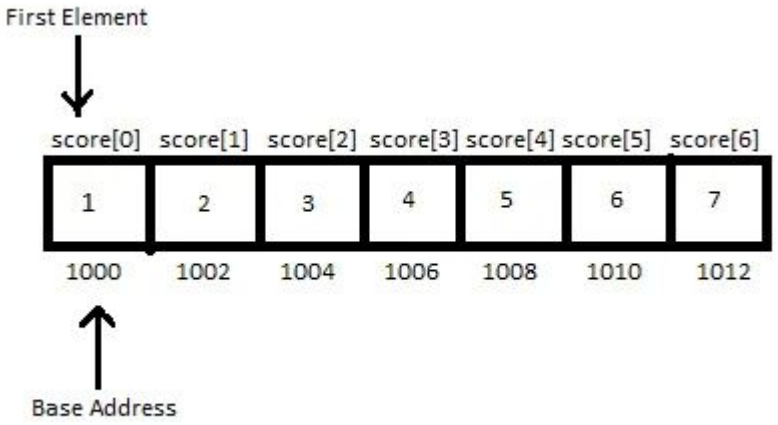
I_Eliminating_Ballons.cpp > main()

```cpp
#include <bits/stdc++.h>    #include errors detected

using namespace std ;
const int tam = 6 * 1e6;
int pos[tam];
int altura[tam];
void solve()
{
   int n , m;
   cin >> n ;
   int count = 0;
   for(int i = 1 ; i ⩽ n ; i++)
   {
      cin >> pos[i];
   }

    for(int i = 1 ;i ⩽ n ;i++)
    {
        if(altura[pos[i]])
        {
            altura[pos[i]]--;
            altura[pos[i]-1]++;
        }else{
            count++;
            altura[pos[i]-1]++;
        }
    }
    cout << count <<'\n';
}

int main()
{
    ios::sync_with_stdio(0); cin.tie(0);
        solve();
}
```

K Battleship

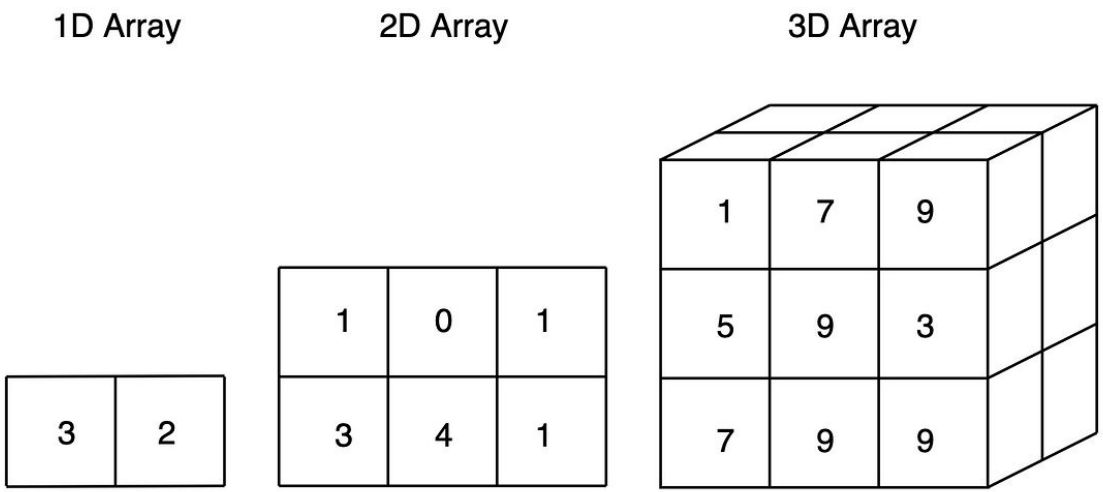
```cpp
#include <bits/stdc++.h>    #include errors detectec

using namespace std ;

bool arr[20][20];
void solve()
{
    int n , d , l , r ,c ;
    cin >> n ;
    bool possible = true;
    while(n--)
    {
        cin >>d >>l>>r>>c;
        for(int i = 0 ; i < l ; i++){
            if(r > 10 || c > 10 || arr[r][c])
            {
                possible = false;
                break;
            }
            arr[r][c] = true;

            if(d){ //vertical
                r++;
            } else { //horizontal
                c++;
            }
        }
    }
    cout << (possible ? "Y" : "N");
}

signed main()
{
    ios::sync_with_stdio(0); cin.tie(0);
        solve();
}
```

ARREGLOS Y VECTORES

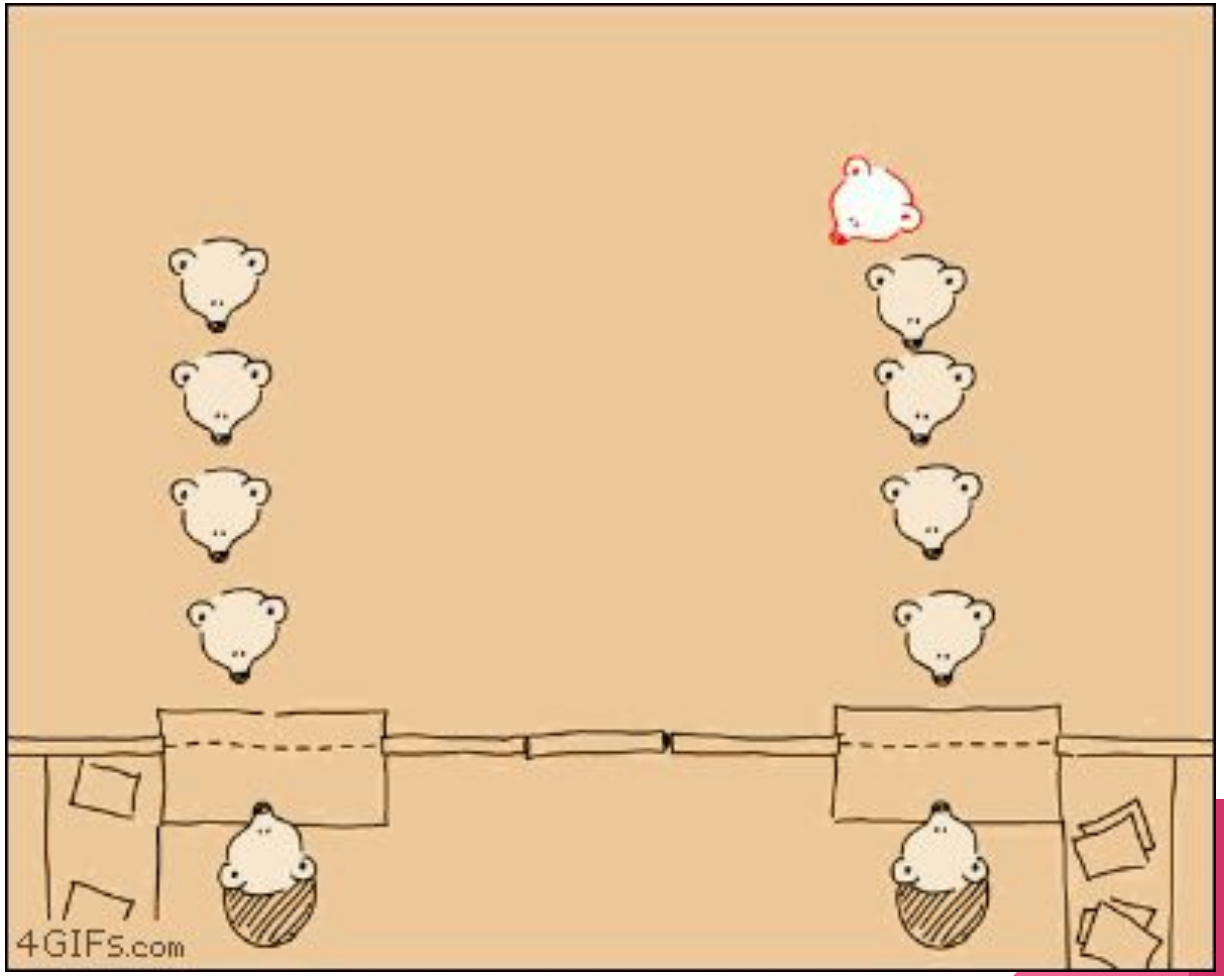


```
vector<int> v;        // []
v.push_back(10);      // [10]       inserta al final
v.push_back(45);      // [10,45]
v[0] = 5;             // [5 ,45]    modifica en indice
v.pop_back();         // [5]        elimina el ultimo
v.size();             // 1          tamano
```

ARREGLOS Y VECTORES

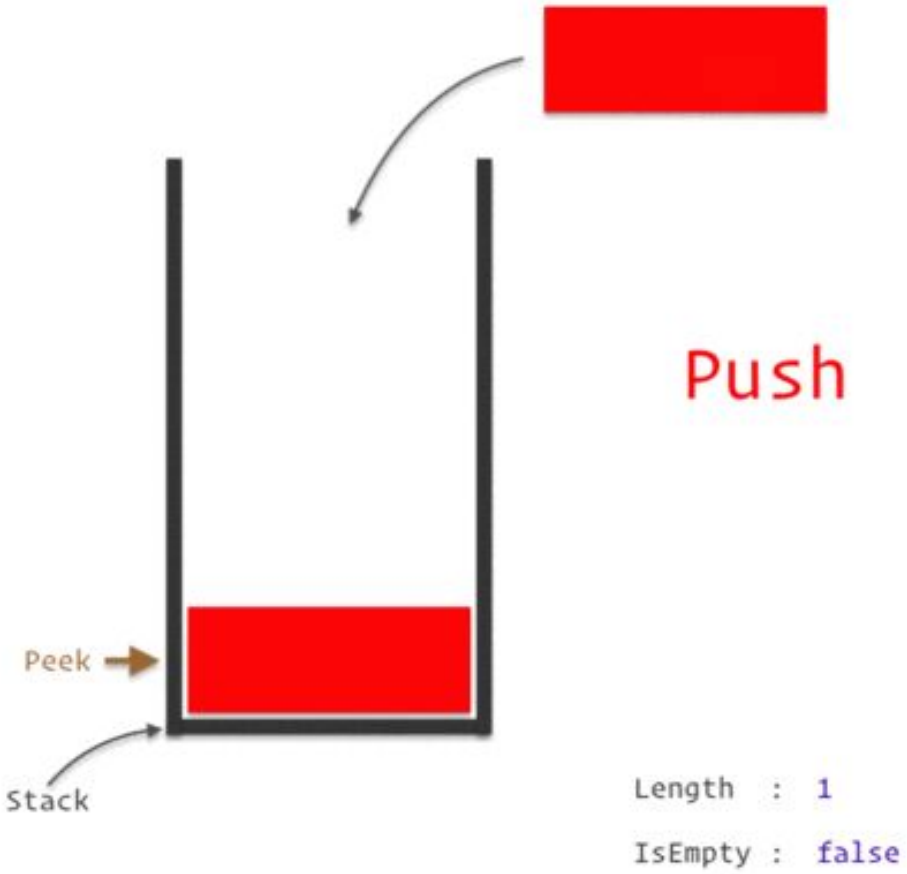


COLA



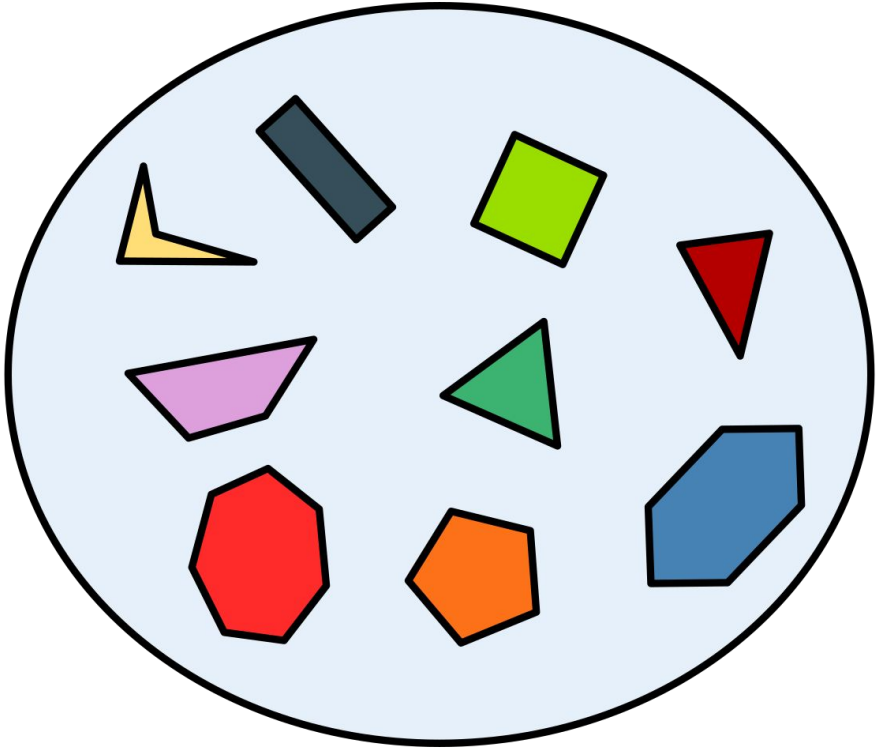
```
queue<int> q;        // []
q.push(1);           // [1]        agrega a la cola
q.push(5);           // [1,5]
q.push(3);           // [1,5,3]
q.push(7);           // [1,5,3,7]
q.size();            // 4
q.front();           // 1          muestra primero en cola
q.pop();             // [5,3,7]    saca primero en cola
q.front();           // 5
q.pop();             // [3,7]
```

PILA



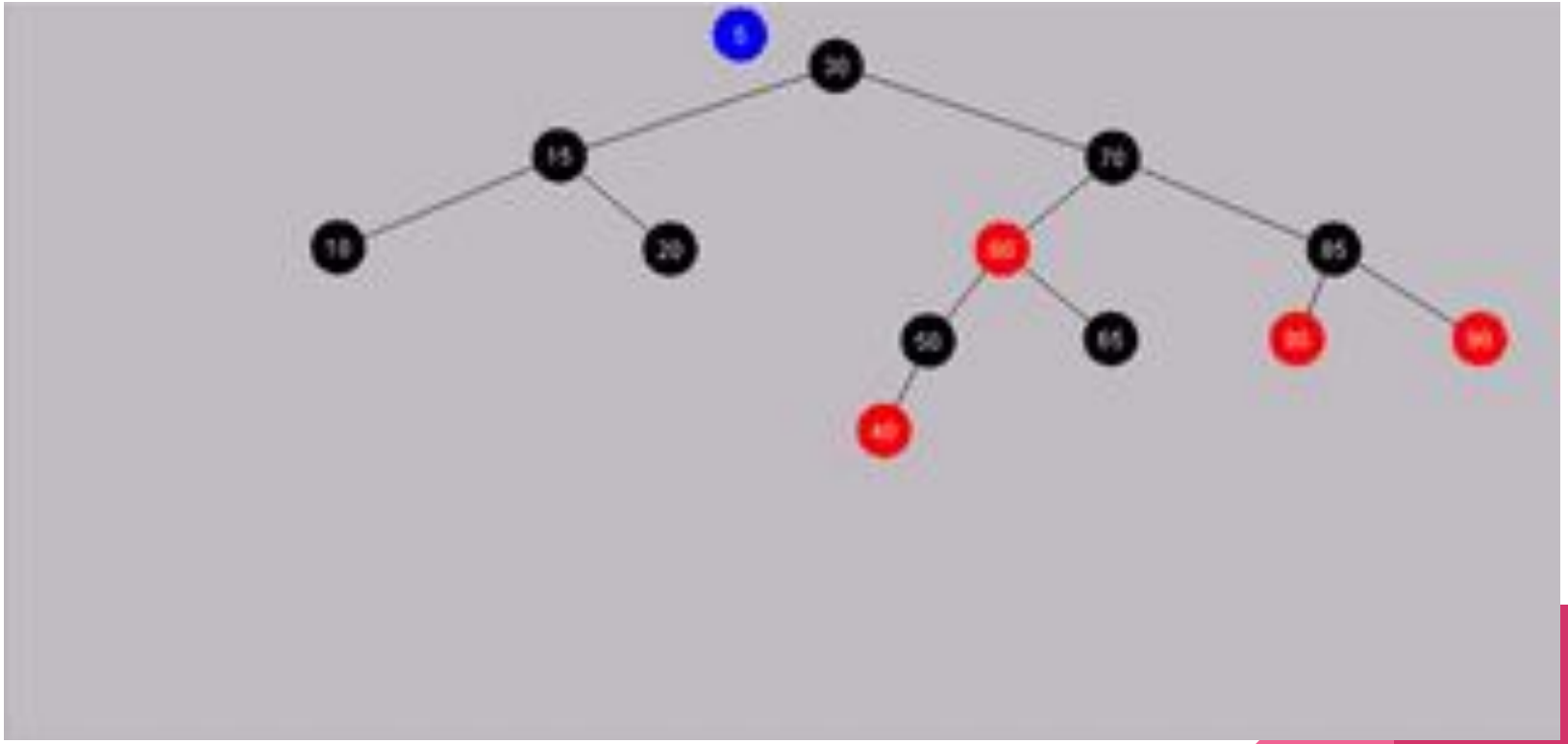
```
stack<int> s;        // []
s.push(1);           // [1]          agrega al tope
s.push(2);           // [1,2]
s.push(3);           // [1,2,3]
s.push(4);           // [1,2,3,4]
s.push(7);           // [1,2,3,4,7]
s.size();            // 5
s.top();             // 7            muestra el tope
s.pop();             // [1,2,3,4]    elemina el tope
s.top();             // 4
s.pop();             // [1,2,3]
```

SET Y MAP



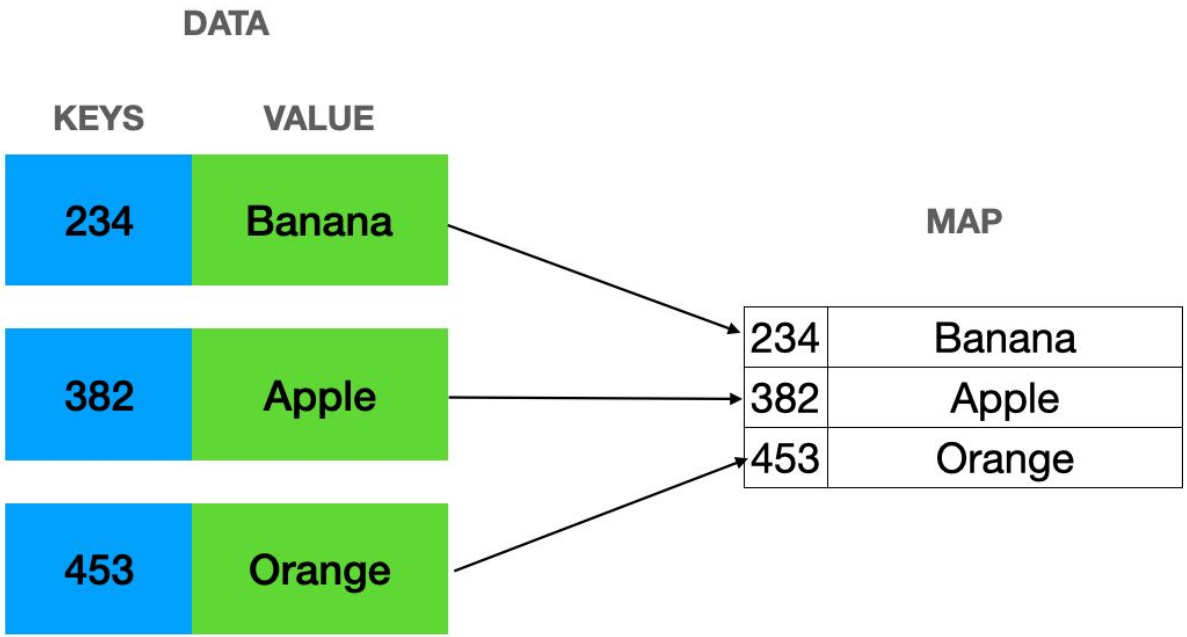
SET Y MAP

SET Y MAP



```
 set<int>st;           // {}
 st.insert(45);        // {45}
 st.insert(3242);      // {45,3242}
 st.insert(56);        // {45,56,3242}
*st.find(3242);        // 3242  retorna un iterador, acceder con (*iterador)
 st.count(56);         // 1
 st.count(13);         // 0
*st.lower_bound(45);   // 45    retorna un iterador, acceder con (*iterador)
 st.erase(45);         // {56,3242}
*st.lower_bound(45);   // 56      elemento mayor o igual
*st.upper_bound(56);   // 3242    elemento mayor
 st.size();            // 2

 auto it = st.upper_bound(56);        // iterador
 int value = *it;                     // 3242
 cout << value ;
```

SET Y MAP


DATA
KEYS
VALUE
234
Banana
382
Apple
453
Orange
MAP
234
Banana
382
Apple
453
Orange

```
 map<int,string> mp;                    // {}
 //———————————————————
 mp.insert({45,"BS"});                  // {<45,BS>}
 mp.insert(make_pair(45,"BS"));         // {<45,BS>}
 mp[45] = "BS";                         // {<45,BS>}
 // tres formas de insertar, todas funcionan igual
 //———————————————————

 mp[3242] = "Red";                      // {<45,BS>,<3242,Red>}
 mp[56] = "Black";                      // {<45,BS>,<56,Black>,<3242,Red>}
 mp[3242] = "Tree";                     // {<45,BS>,<56,Black>,<3242,Tree>}
 mp.count(56);                          // 1
 mp.count(13);                          // 0
*mp.lower_bound(45);                    // <45,BS>
 mp.erase(45);                          // {<56,Black>,<3242,Tree>}
*mp.lower_bound(45);                    // <56,Black>
*mp.upper_bound(56);                    // <3242,Tree>
 mp.size();                             //  2

 auto it = mp.upper_bound(56);           // <3242,Tree>
 int key = it→first;                     // 3242
 string value = it→second;               // Tree
```